

Adaptive Ticket Routing through Retrieval-Augmented In-Context Learning from Analyst Behaviour at CERT-EU

Alex Diaz-Santos*

alejandro.diazsantos@cert.europa.eu
CERT-EU, European Commission
Brussels, Belgium

Krzysztof Socha

krzysztof.socha@cert.europa.eu
CERT-EU, European Commission
Brussels, Belgium

Abstract

Ticket routing is a critical operational task: organisations must triage incoming requests to specialised teams, typically via queues, but routing policies are organisation-specific and evolve continuously. Rule-based systems demand constant maintenance, while supervised classifiers suffer from concept drift and require periodic retraining. These challenges intensify in regulated environments where sensitive data prevents cloud-based AI services, annotation capacity is limited, and misrouting carries significant cost.

We present an adaptive ticket routing system that frames routing as retrieval-augmented in-context learning with *temporal feedback learning*. Rather than training a static model, the system retrieves similar successfully routed tickets as demonstrations, grounding predictions in accumulated organisational expertise. It learns continuously from routine analyst behaviour: corrections are captured as error signals, while predictions unchanged beyond a calibrated time threshold are interpreted as implicit confirmation—transforming everyday workflow into supervision at zero incremental annotation cost. Predictions are constrained to queues present in retrieved demonstrations; an on-premises LLM resolves conflicts only when demonstrations disagree.

Deployed at CERT-EU¹, the system operates in two modes: *assisted routing*, where analysts receive recommendations based on similar historical cases (2× speedup); and *selective autonomy*, where tickets assigned to high-confidence queues are routed automatically (22× speedup). Over six months of production deployment (1900 tickets), it achieves 86.3% weighted F1, outperforming supervised baselines by 4.5 points and zero-shot LLM inference by 15 points. The system runs entirely on-premises and adapts continuously without retraining or dedicated annotation, offering a practical blueprint for organisations operating under strict data-sovereignty constraints.

CCS Concepts

- Information systems → Information systems applications;
- Computing methodologies → Artificial intelligence.

*Main author

¹CERT-EU is the cybersecurity service provider for European Union entities.



This work is licensed under a Creative Commons Attribution 4.0 International License. KDD '26, Jeju Island, Republic of Korea

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2259-2/2026/08

<https://doi.org/10.1145/3770855.3818313>

Keywords

In-Context Learning, Retrieval-Augmented Classification, Continual Adaptation, Security Operations, Human-in-the-Loop

ACM Reference Format:

Alex Diaz-Santos and Krzysztof Socha. 2026. Adaptive Ticket Routing through Retrieval-Augmented In-Context Learning from Analyst Behaviour at CERT-EU. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26)*, August 09–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3770855.3818313>

1 Introduction

Ticket routing is a critical operational challenge across service-oriented organisations: incoming requests must be triaged and assigned to the appropriate specialised team based on content, urgency, and required expertise. Whether in IT helpdesks, customer support centres, or cybersecurity operations, incorrect routing delays resolution, wastes analyst time, and degrades service quality. A *ticket*—a structured record of a request, issue, or event requiring action—typically arrives in a shared inbox and must be dispatched to the correct *queue* (a work assignment category representing a specific team or functional area), where a specialised team processes it according to established workflows.

This paper presents a system deployed at CERT-EU, where analysts process hundreds of tickets daily from diverse sources including functional mailboxes, threat intelligence feeds, and partner organisations. These tickets encompass cybersecurity attacks, threat intelligence reports, vulnerability disclosures, and security advisories. A critical challenge is routing each ticket to the appropriate queue—such as threat intelligence queues for analysis, incident response queues for active incidents, or information sharing queues for partner notifications—based on content, severity, and required expertise. Incorrect routing delays response, potentially leaving systems exposed to threats.

Current approaches rely on manual triage, rule-based systems, or static machine learning classifiers. Rule-based systems require constant maintenance as organisations evolve. Static ML approaches suffer from concept drift and require periodic retraining cycles—typically 2–3 months involving data collection, annotation, model training, validation, and deployment [1, 9]. This operational overhead makes them impractical for rapidly evolving environments where routing policies change faster than retraining cycles complete. Both fail to capture institutional knowledge as analyst expertise evolves.

Large language models (LLMs) offer promising capabilities for text classification tasks, but present challenges in regulated environments. Cloud-based LLM services often cannot be used due to data

sovereignty requirements for handling sensitive ticket information. Moreover, LLMs applied in zero-shot mode lack the organisational context needed for accurate routing—they cannot know that a particular organisation routes partner notifications to specific queues or that certain ticket types require specialist attention. Few-shot prompting with static examples partially addresses this, but fails to capture evolving organisational nuances: routing conventions shift over time, new ticket types emerge, and the same content may warrant different routing depending on current operational priorities. This organisational knowledge exists only in the accumulated decisions of experienced analysts—and it evolves continuously.

We address these challenges through retrieval-augmented in-context learning (ICL) along with *temporal feedback learning*: rather than relying on static training or general LLM reasoning, we dynamically retrieve similar previously routed tickets as demonstrations that encode institutional routing expertise. The system learns continuously from routine analyst behaviour—corrections are captured as error signals, while predictions unchanged beyond a calibrated time threshold are interpreted as implicit confirmation. This approach enables continuous adaptation without retraining, grounds decisions in organisational practices, and operates entirely on-premise. While our deployment is in cybersecurity, these challenges apply broadly to healthcare, legal services, government, and other domains handling sensitive data.

1.1 Contributions

- (1) *Temporal feedback learning*: Our primary methodological contribution is a novel mechanism that transforms routine analyst workflow into continual model improvement. Unlike active learning which requires explicit labelling sessions, or weak supervision which relies on heuristic rules, we extract learning signals from the analyst interventions: corrections are captured as error signals, while predictions unchanged beyond a calibrated time threshold are interpreted as implicit confirmation. This temporal signal—measuring *if* analysts correct rather than *what* they click—enables perpetual adaptation at zero incremental annotation cost beyond analysts' existing routing workflow.
- (2) *Retrieval-augmented in-context learning for classification*: We adapt in-context learning [2, 6] to ticket routing by dynamically retrieving similar previously routed tickets as demonstrations. This architecture *eliminates retraining entirely*: the system adapts by adding new demonstrations to organisational memory rather than updating model parameters. No periodic retraining cycles, no annotation campaigns, no model validation—the system evolves continuously as analysts route tickets. Unlike standard RAG focused on text generation, we constrain output to queues present in retrieved demonstrations, providing a structural guarantee against invalid predictions.
- (3) *Phased deployment with empirical calibration*: We contribute a reusable deployment pattern for AI systems in high-stakes environments: bootstrap → assisted routing → selective autonomy. Our empirical finding that automation scope must

be determined through production observation provides actionable guidance for practitioners deploying similar systems.

- (4) *Production validation*: Six months deployment at CERT-EU achieving 86.3% F1 and 5.5× speedup, demonstrating that retrieval-augmented ICL outperforms both traditional ML (+4.5pp vs. XGBoost) and parameter-efficient fine-tuning (+2.9pp vs. Low-Rank Adaptation) while adapting continuously without retraining.

1.2 Paper organisation

The remainder of this paper is organised as follows. Section 2 reviews related work. Section 3 presents the system design, covering the classification pipeline, temporal feedback learning, and operating modes. Section 4 evaluates the system through comparison with alternatives, classification performance, deployment strategy, and operational impact. Section 5 discusses implications and limitations. Section 6 concludes with key findings and future directions.

2 Related work

Ticket Routing. Traditional approaches use rule-based systems or batch-trained classifiers [5, 13, 22]. Recent transformer-based methods achieve strong performance on ticket automation [38] and issue classification [33], but require periodic retraining. Recommendation systems suggest queues without autonomous decisions [4]. Our work makes autonomous decisions for high-confidence queues while continuously adapting through temporal feedback.

In-Context Learning (ICL). ICL enables LLMs to perform tasks by conditioning on demonstrations without parameter updates [2, 6]. Demonstration selection significantly impacts performance [19, 20, 23], motivating retrieval-based approaches [3, 18, 28]. While ICL offers flexibility over fine-tuning for classification [7], our work extends this paradigm with continual adaptation from implicit temporal feedback.

Retrieval-Augmented Systems. RAG combines vector search with LLMs for knowledge-intensive tasks [12, 14, 17]. Hybrid retrieval combining dense and sparse methods outperforms single approaches [24, 29]. Unlike standard RAG for generation, we retrieve demonstrations for classification with constrained outputs—the model selects only from queues present in retrieved demonstrations.

Continual Learning. Continual learning adapts models to non-stationary distributions without catastrophic forgetting [9, 16, 35], with recent work addressing LLMs [31] and concept drift [8]. Our retrieval-based architecture sidesteps forgetting entirely: new demonstrations are *added* to memory rather than modifying representations. We use “continual adaptation” rather than “continual learning” as our system updates memory, not parameters.

Implicit Feedback and Human-in-the-Loop (HITL). Weak supervision extracts labels from programmatic sources [25, 26]. HITL approaches improve models through human feedback [21, 36, 37], with active learning minimising annotation [30] and coactive learning using user edits as preference signals [32, 34]. Our temporal feedback differs: it requires *no explicit action*—analysts perform

normal routing while the system passively observes *if* corrections occur, extracting learning signals from routine operations.

3 System design

The core insight is that resolved tickets constitute *organisational memory*: each ticket-queue pair demonstrates how the organisation routes specific ticket types. This accumulated expertise captures institutional knowledge that no static classifier can replicate.

We frame routing as retrieval-augmented in-context learning. When a new ticket arrives, we retrieve the K most similar resolved tickets. These serve as demonstrations showing how similar cases were handled. The model then applies the routing pattern from these demonstrated examples to the new ticket—leveraging in-context learning [2] with demonstrations dynamically selected from evolving organisational memory rather than statically defined.

This approach provides three advantages: (1) Dynamic adaptation without retraining—new demonstrations enter as analysts resolve tickets; (2) Evidence-grounded classification—constraining output to queues present in retrieved demonstrations prevents invalid queue predictions; (3) Interpretable decisions—analysts can inspect demonstrations underlying any prediction.

3.1 Classification pipeline

Tickets are ingested from the source system via a fetcher service and queued through Kafka for processing. Once a ticket reaches the core pipeline (Figure 1), classification proceeds in three stages:

Step 1: Multi-signal retrieval. We retrieve K similar resolved tickets from organisational memory using three complementary signals:

- *Dense embeddings* capture semantic similarity—matching “ransomware attack” with “file encryption threat”
- *Sparse embeddings* (BM25 [27], a probabilistic lexical ranking function) capture exact keywords—vulnerability identifiers, IP addresses, malware names
- *Late interaction* (ColBERT [15]) enables token-level matching for partial overlap and positional relationships

The retrieval operates in two stages. First, we prefetch top 20 candidates using a weighted combination of dense and sparse scores:

$$\text{score}_{\text{prefetch}}(q, c) = \alpha \cdot \text{sim}_{\text{dense}}(q, c) + \beta \cdot \text{sim}_{\text{sparse}}(q, c) \quad (1)$$

The reranker then scores these candidates using late interaction for fine-grained token-level matching, and the top K demonstrations are selected for classification.

Step 2: Queue prediction. Let $Q = \{q_1, q_2, \dots, q_K\}$ be the queues assigned to the K retrieved demonstrations. The prediction strategy depends on demonstration agreement:

- *Consensus case*: When all demonstrations share the same queue ($|Q| = 1$), we assign that queue directly. Unanimous demonstrated evidence provides high confidence without requiring LLM inference.
- *Conflict case*: When demonstrations disagree, we employ an on-premise LLM to synthesise the evidence. The LLM receives the new ticket, K demonstrations with their queues and similarity scores, and selects the most appropriate queue

from those present in the demonstrations (see Appendix D for full prompt template).

Step 3: Output constraint and validation. The system outputs the predicted queue, constrained to queues present in retrieved demonstrations. We prevent invalid queue outputs through: (1) prompt listing only allowed queues from retrieved demonstrations, (2) post-processing validation against the allowed queue set, (3) fallback to highest-similarity demonstration’s queue if the LLM output does not match any allowed queue. In 6+ months of operation, no invalid queue outputs have been observed—the system has never routed to a non-existent queue. This is a structural guarantee of the constrained output design rather than a property of the LLM itself.

Organisational memory. A vector database stores all resolved tickets with analyst-assigned queues, indexed with all three embedding types. The database grows continuously as analysts resolve tickets, creating a virtuous cycle: accurate predictions lead to fewer corrections, more confirmations enter the database, and retrieval quality improves further.

3.2 Temporal feedback learning

Traditional machine learning systems require explicit annotation: humans must label examples, often through dedicated labelling interfaces separate from their normal workflow. Active learning reduces annotation volume but still requires explicit labelling sessions. Our temporal feedback learning eliminates this overhead entirely by reinterpreting routine analyst work as implicit labelling. Figure 2 provides an overview of our temporal feedback mechanism.

The key insight is that analyst *timing* carries information. When analysts quickly correct a routing prediction, this signals an error. When analysts leave a prediction unchanged for extended periods despite regular queue review, this signals implicit validation. By observing these temporal patterns, we extract learning signals from normal workflow without any additional analyst effort.

Implementation. When predicting a queue, we store the prediction in Redis with 10-day TTL (Time To Live), creating a temporal observation window:

- *Early corrections (Days 0–7)*: Analyst moves ticket to different queue $\rightarrow$ index as correction signal with analyst’s choice
- *Sustained predictions (Day 8+)*: Ticket remains in predicted queue $\rightarrow$ interpret as implicit validation, index as positive demonstration
- *Inbox returns*: Analyst returns to generic Inbox $\rightarrow$ continue monitoring until definitive assignment

The 7/8-day thresholds were calibrated based on internal analysis of historical routing patterns: the majority of analyst corrections occur within 7 days of initial assignment (see Appendix B for threshold sensitivity analysis). Formally, given a prediction p at time t_0 and analyst action a at time t_a :

$$\text{label}(p) = \begin{cases} \text{correction}(a) & t_a - t_0 \leq 7\text{d}, a \neq p \\ \text{confirmed}(p) & t_a - t_0 > 7\text{d}, a = p \\ \text{pending} & \text{otherwise} \end{cases} \quad (2)$$

This formulation captures both rapid error signals and delayed validation signals from normal workflow.

Dual-signal learning. Learning from both corrections and confirmations is critical. Corrections alone would bias the system toward recent errors. Confirmations provide positive reinforcement that stabilises correct prediction patterns, enabling the system to reinforce its successes while adapting to corrections.

Preventing catastrophic forgetting. Unlike gradient-based continual learning methods that risk forgetting [9, 16], our retrieval-based architecture uses additive learning: each ticket adds to the database rather than modifying existing demonstrations. New patterns don't overwrite old patterns—they coexist in organisational memory.

3.3 Operating modes

The system operates in two distinct modes:

- **Assisted routing:** The system provides a recommendation in the ticket management interface, but the analyst makes the final routing decision. Used for all tickets during initial deployment and for lower-confidence queues in mature deployment.
- **Selective autonomy:** The system automatically routes the ticket to the predicted queue without requiring analyst intervention. Used only for high-confidence queues after empirical validation. Analysts retain override authority and can correct any prediction.

4 Evaluation

4.1 Experimental setup

Queue structure. The organisation structures ticket handling into specialised queues reflecting operational responsibilities. Queue names have been anonymised for confidentiality; we use generic identifiers (Q1–Q11) throughout. We evaluate on 11 queues spanning five functional areas:

- *Threat Intelligence:* Q7 handles threat analysis, advisories, and intelligence products
- *Incident Response:* Q9 handles active security tickets requiring investigation and remediation
- *Information Sharing:* Q1–Q5 handle notifications and reports from partner organisations, threat intelligence feeds, and coordination networks
- *Operations:* Q6 handles infrastructure-related tickets; Q10 handles cross-border coordination with external entities; Q11 handles vulnerability disclosure and penetration testing coordination
- *Spam:* Q8 captures noise—messages with no operational value

New tickets initially arrive in a generic Inbox queue, from which analysts triage them to specialised queues. The system's goal is to accelerate this triage by predicting the appropriate specialised queue.

The queue structure exhibits topical overlap that increases classification difficulty—a vulnerability disclosure may legitimately route to the threat intelligence queue (for analysis), an information sharing queue (for notification), or the incident response queue (for remediation) depending on context and organisational priorities.

Evaluation methodology. We evaluated system performance on 1900 tickets sampled to match production queue distribution, processed during months 5–6 (mature system operation). For each ticket, we measure:

- *Classification accuracy:* Comparing system predictions against the final analyst-assigned queue (ground truth)
- *Routing velocity:* Time from ticket creation to queue assignment

For velocity comparison, we compare against a manual baseline derived from historical routing times when analysts operated without ML assistance, enabling direct measurement of the system's operational impact.

System configuration. $K = 5$ similar tickets retrieved per query; nomic-embed-text (768d) for dense embeddings, BM25 for sparse embeddings, ColBERT for late-interaction reranking; Qwen3:32B via llama.cpp for conflict resolution, deployed entirely on-premise. See Appendix A for full configuration, hyperparameters, and hardware details.

Alternative approaches compared. To validate our retrieval-augmented approach, we compared against several baselines. All supervised approaches (XGBoost, K-Nearest Neighbours, LoRA fine-tuning) were trained on the same 12000 historical tickets used to bootstrap our system's organisational memory, ensuring a fair comparison. Evaluation was performed on the same held-out set of 1900 tickets for all methods:

- *Zero-shot LLM:* Qwen3:32B classifying tickets without demonstrations—isolating the contribution of retrieved examples
- *Few-shot LLM (static):* Qwen3:32B with 5 fixed, manually-selected examples per queue—isolating whether static demonstrations suffice
- *KNN + semantic embeddings:* K-nearest neighbours with majority voting—isolating whether LLM reasoning adds value beyond retrieval
- *XGBoost + TF-IDF (Term Frequency-Inverse Document Frequency):* Traditional ML classifier with lexical features—representing conventional supervised approaches
- *XGBoost + semantic embeddings:* ML classifier with dense embeddings—isolating whether retrieval adds value beyond embedding quality
- *LoRA fine-tuned LLM:* Qwen3:32B fine-tuned on 12000 historical tickets using parameter-efficient fine-tuning (LoRA) [10, 11]—representing state-of-the-art supervised classification requiring periodic retraining

We selected retrieval-augmented in-context learning over these alternatives because: (1) it requires no periodic retraining as organisational practices evolve, (2) it adapts continuously through the temporal feedback mechanism, and (3) it provides interpretable decisions through inspectable demonstrations.

Metrics. Precision, recall, F1 per queue; weighted aggregates by support.

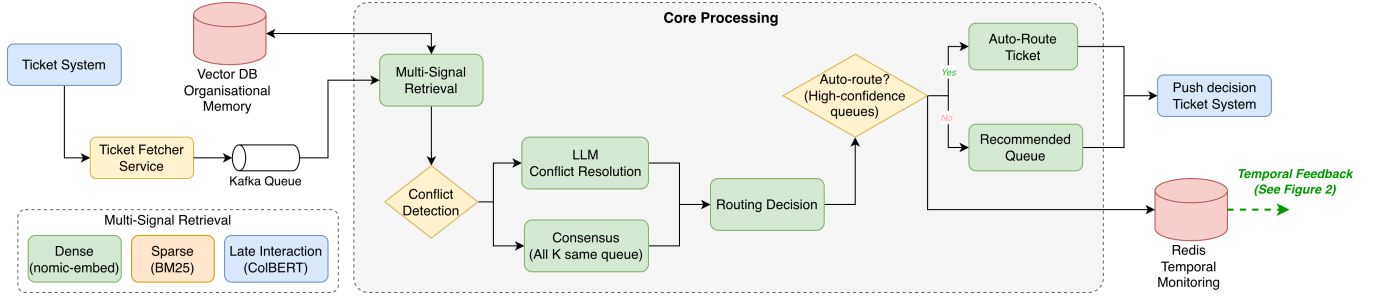


Figure 1: System architecture. New tickets are embedded and matched against K similar demonstrations from organisational memory. Unanimous demonstrations yield direct prediction; conflicts trigger LLM resolution. Predictions are stored in Redis for temporal feedback.

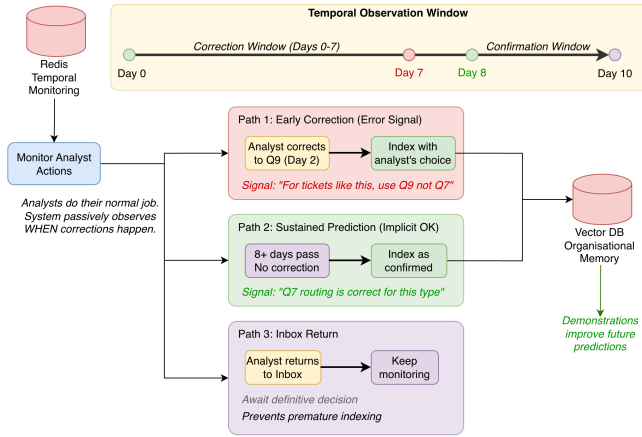


Figure 2: Temporal feedback learning. Early corrections (Days 0–7) generate error signals; sustained predictions (Day 8+) are interpreted as implicit confirmation. Both signals update organisational memory at zero annotation cost.

Table 1: Comparison with alternative approaches on 1900 tickets. All methods trained/bootstrapped on 12000 historical tickets.

Approach	Prec.	Rec.	F1
<i>Traditional ML (requires retraining)</i>			
KNN + semantic emb.	75.2	74.8	74.9
XGBoost + TF-IDF	77.5	77.8	77.3
XGBoost + semantic emb.	81.8	82.0	81.8
<i>LLM-based approaches</i>			
Zero-shot LLM	73.3	71.1	71.3
Few-shot LLM (static)	76.8	75.4	75.8
LoRA fine-tuned LLM	83.7	83.2	83.4
Ours	86.9	86.1	86.3

4.2 Why retrieval-augmented ICL?

Before deploying our system, we evaluated alternative approaches on the same 1900-ticket dataset to determine the best architecture for our requirements. Table 1 summarises the comparison.

The 15-point gap versus zero-shot LLM demonstrates that demonstrations provide essential organisational context that general LLM reasoning cannot replicate—the model cannot know CERT-EU’s routing conventions without examples. Few-shot prompting with static examples improves over zero-shot (+4.5pp), but the 10.5-point gap versus our approach reveals its limitation: fixed examples cannot capture evolving routing conventions, emerging ticket types, or context-dependent priorities that dynamic retrieval addresses by selecting the most relevant demonstrations for each specific ticket. KNN with semantic embeddings (74.9% F1) confirms that retrieval alone is insufficient; the LLM’s ability to synthesise evidence from multiple demonstrations adds significant value.

Even LoRA fine-tuning [11], representing state-of-the-art parameter-efficient fine-tuning, achieves only 83.4% F1—nearly 3 points below our approach. While LoRA reduces computational costs compared to full fine-tuning, it still requires periodic retraining as organisational practices evolve: when new queue types emerge or routing conventions change, the fine-tuned model must be retrained on updated data. This retraining overhead is precisely what we sought to avoid. Our retrieval-augmented approach achieves superior accuracy while adapting continuously through the temporal feedback mechanism without any model parameter updates—new demonstrations are simply added to organisational memory. All pairwise differences are statistically significant (McNemar’s χ^2 test, $p < 0.001$; see Appendix C).

4.3 Classification performance

Using our retrieval-augmented system, we evaluated classification accuracy on the 1900-ticket dataset. Table 2 shows per-queue performance. The system achieves 86.1% accuracy (1636/1900 correct), with Precision=86.9%, Recall=86.1%, Weighted F1=86.3%. Higher precision than recall indicates conservative behaviour appropriate for production deployment.

Q1 and Q2 achieve perfect F1=1.000. We attribute this to the highly standardised nature of messages routed to these queues: both receive notifications from well-established external sources that produce messages following well-defined templates with consistent subject line patterns, structured formatting, and recognisable sender signatures. This template consistency means retrieved demonstrations are near-exact matches for incoming tickets, enabling reliable classification. Perfect scores on structured-template

Table 2: Per-queue classification performance of the retrieval-augmented system on 1900 evaluation tickets. Queues ordered by F1 score; queues with $F1 > 0.86$ qualified for selective autonomy. Queue names anonymised for confidentiality.

Queue	Support	Precision	Recall	F1
Q1	130	1.000	1.000	1.000
Q2	71	1.000	1.000	1.000
Q3	89	0.976	0.988	0.982
Q4	9	0.889	1.000	0.941
Q5	197	0.913	0.918	0.916
Q6	193	0.839	0.961	0.896
Q7	280	0.839	0.904	0.870
Q8	550	0.938	0.804	0.866
Q9	278	0.820	0.795	0.807
Q10	73	0.650	0.765	0.703
Q11	30	0.548	0.607	0.576

Table 3: Classification performance by prediction path on 1900 evaluation tickets.

Path	n	F1
Consensus ($K=5$ unanimous, no LLM)	1064	0.944
Conflict (LLM-resolved)	836	0.760

queues should not be interpreted as indicating perfect classification ability in general—these queues represent a favourable case where lexical and structural similarity strongly predict the correct routing decision.

Prediction path analysis. The classification pipeline follows two paths depending on demonstration agreement (Section 3.1). Table 3 shows performance by prediction path. Of the 1900 evaluation tickets, 56% (1064) followed the consensus path where all retrieved demonstrations agreed unanimously, bypassing LLM inference entirely; the remaining 44% (836) required LLM conflict resolution.

Both paths are grounded in retrieved demonstrations—consensus is an efficiency optimisation when demonstration agreement makes LLM reasoning unnecessary, not a different paradigm. The consensus path achieves substantially higher F1 (0.944 vs. 0.760), confirming that unanimous demonstration agreement is a strong signal for classification confidence. This design also enables rapid adaptation to new queues: when Q12 was created with zero historical precedent, demonstrations accumulated through temporal feedback until consensus emerged naturally, reaching $F1=0.79$ within three weeks (Table 5), without retraining or manual intervention.

These per-queue results directly informed our selective autonomy decisions, as described in the following section.

4.4 Deployment strategy

Deploying AI systems in high-stakes operational environments requires building trust incrementally. We adopted a three-phase deployment strategy that progressively increased automation as the system demonstrated reliability.

Phase 1: Bootstrapping (Week 1). We initialised the vector database with approximately 12000 resolved tickets from the preceding year, establishing initial organisational memory. This one-time setup used existing historical data—tickets already processed and queued by analysts—requiring no new annotation.

Phase 2: Assisted routing (Months 1–3). Deployment began in recommendation-only mode. For every ticket, the system populated a RecommendedQueue field visible to analysts who retained full decision authority. This phase served dual purposes: (1) collecting performance metrics to identify high-reliability queues, and (2) building analyst trust through transparent, inspectable recommendations. Analysts could accept recommendations with a single click or override with their own judgement—either action contributing to the temporal feedback learning loop.

Phase 3: Selective autonomy (Months 4–6). Based on classification performance (Table 2), we enabled autonomous routing for queues meeting strict criteria: $F1 > 0.86$, consistent performance across time, and lower misrouting consequences. Six queues qualified: Q1–Q5 (information sharing) and Q7 (threat intelligence). These share characteristics making autonomous routing appropriate: well-defined patterns, high historical volume, and informational content rather than active incidents requiring immediate analyst judgement. Notably, Q6 and Q8 were excluded despite exceeding the F1 threshold due to operational risk: Q6 handles infrastructure tickets where misrouting delays critical maintenance, and Q8 misclassification risks discarding legitimate tickets. Q9 handles active security incidents requiring immediate analyst judgement. Lower-performing queues (Q10, Q11) remain in assisted routing mode.

Empirical calibration. The decision of which queues qualify for autonomous routing was determined empirically through production observation rather than assumed from offline metrics alone. Broader queues exhibited higher variability, leading us to restrict autonomous routing to the six high-confidence queues with consistent performance.

4.5 Operational impact

We measure operational impact through routing velocity: the elapsed time from ticket creation to assignment to a specialised (non-Inbox) queue. This metric captures the end-to-end benefit of the system, reflecting both classification accuracy and workflow integration.

Routing velocity results. Table 4 shows routing velocity for the 1900 evaluation tickets compared against the manual baseline (historical routing time without ML assistance). Each speedup factor is computed as the ratio of manual baseline median to the respective deployment phase median.

The 22 $\times$ speedup for selective autonomy reflects the ratio of manual baseline median (22.76 min) to selective autonomy median (1.03 min)—i.e., tickets that the system routes autonomously with high confidence are assigned in near-minute time compared to the ~23 minutes typical of fully manual triage. This comparison captures the end-to-end operational benefit: time from ticket arrival to queue assignment under each regime.

Table 4: Routing velocity comparison. Manual baseline represents historical routing time without ML assistance.

Condition	Share	Median (min)	Speedup
Manual Baseline	—	22.76	—
With System (combined)	100%	4.15	5.5×
– Assisted Routing	73.6%	11.57	2×
– Selective Autonomy	26.4%	1.03	22×

Table 5: Adaptation to new Q12 queue over three weeks. Support shows cumulative demonstrations in organisational memory. No retraining or annotation required—the system learned entirely from analyst routing decisions via temporal feedback.

Week	Support	F1	Notes
1	~15	0.32	Insufficient data
2	~35	0.58	Recognising patterns
3	~50	0.79	Reliable recommendations

The results demonstrate the value of graduated autonomy. Assisted routing alone delivers 2× speedup—analysts accept recommendations faster than formulating routing decisions independently. Selective autonomy achieves 22× speedup for qualifying tickets, with near-minute median routing that eliminates triage bottlenecks for predictable ticket types.

4.6 Rapid adaptation to organisational change

A key advantage of retrieval-augmented learning is rapid adaptation to new queues without retraining. During the evaluation period, a new queue (Q12) was created to handle tickets related to cybersecurity exercises and simulations—a queue that had no historical precedent in the system.

Initially, the system had zero performance for this queue since no demonstrations existed. However, as analysts began routing exercise-related tickets to the new queue, the system rapidly learned to recognise these patterns (Table 5).

The key insight is that the system required no manual intervention—as analysts simply did their normal job of routing tickets, the temporal feedback mechanism automatically incorporated their decisions into organisational memory. Within three weeks, the system began reliably recommending Q12 for relevant tickets.

For comparison, traditional ML approaches would require: (1) manual annotation of 100+ examples, (2) feature engineering or fine-tuning, (3) model retraining and validation, and (4) deployment testing—typically a 2–3 month cycle. The retrieval-based approach achieves reasonable performance in weeks with zero additional labelling overhead.

This rapid adaptation capability is operationally significant: security organisations frequently restructure queues in response to evolving threat landscapes, and systems requiring lengthy retraining cycles become impediments to organisational agility.

4.7 Error analysis and semantic cohesion

Semantic cohesion. We measure intra-queue semantic cohesion as a proxy for routing consistency. For a queue q containing tickets $\{e_1, e_2, \dots, e_n\}$ with dense embeddings, we define:

$$\text{cohesion}(q) = \frac{2}{n(n-1)} \sum_{i < j} \cos(e_i, e_j) \quad (3)$$

i.e., the average pairwise cosine similarity among all ticket embeddings assigned to the same queue.

We compared cohesion between historical analyst decisions (pre-deployment) and system-assisted decisions. Average cohesion improved from 57.8% to 60.3% (+2.5 percentage points). Higher cohesion indicates that tickets within each queue share stronger thematic similarity, reflecting more consistent application of routing criteria than variable human decisions made under time pressure. This consistency could reduce downstream re-routing when analysts review their queues and discover misplaced tickets—a common source of operational friction. We did not measure re-routing rates directly, but investigating this relationship represents a promising direction for future work.

We use cohesion as a *consistency* metric rather than a direct *correctness* metric. Higher cohesion does not necessarily mean better routing in all cases—a queue could have high cohesion by routing only unambiguous cases while misrouting borderline ones. However, combined with stable F1 scores, improved cohesion indicates that the system applies more uniform routing criteria than manual triage under time pressure.

Error analysis. We manually analysed the 264 misrouted tickets (13.9%) to identify systematic patterns:

- *Cross-queue ambiguity:* Tickets legitimately spanning multiple queues. Example: a vulnerability disclosure affecting a partner organisation could route to threat intelligence, information sharing, or incident response. In many cases, the system’s prediction was defensible; analysts chose based on workflow considerations invisible to the model.
- *Organisational transitions:* Accuracy dropped during weeks when new sub-queues were introduced, recovering within 2–3 weeks as demonstrations accumulated. This represents expected adaptation behaviour, not a fundamental system limitation.
- *Insufficient context:* Routing decisions influenced by factors inaccessible to text-based classification: sender reputation, ongoing investigations, or political considerations.
- *Limited demonstrations:* Queues with few demonstrations show degraded performance, though this relationship is non-linear due to varying queue complexity.
- *Inherently ambiguous:* Tickets where expert analysts disagreed on correct routing, representing the irreducible error floor for text-based classification without access to external context.

5 Discussion

Trust and adoption. Analysts accepted the system because they can inspect the demonstrations underlying each prediction—similar historical tickets provide justification that black-box predictions

lack. The three-phase deployment strategy proved essential: starting in recommendation-only mode allowed analysts to calibrate trust while the system accumulated demonstrations. When analysts rotate or leave, their routing patterns persist as indexed demonstrations, preserving institutional knowledge.

Zero incremental annotation cost. Unlike active learning approaches requiring explicit labelling sessions, our temporal feedback mechanism extracts signals entirely from routine workflow. Analysts make routing decisions regardless of whether our system exists—we interpret the *timing* of these decisions to derive examples at zero additional effort. The system requires one-time bootstrapping with ~12000 historical tickets but no ongoing annotation.

Delayed corrections. A concern with the 8-day confirmation threshold: incorrectly routed tickets not corrected within the window would be indexed as positive demonstrations. We mitigate this through: (1) regular queue reviews catching misrouted tickets; (2) autonomous routing restricted to high-confidence queues; (3) periodic manual audits; (4) robustness to noise—isolated errors among thousands of demonstrations have minimal impact on majority-vote predictions.

Terminology. We use “continual adaptation” rather than “continual learning” [16, 35] deliberately: our system updates *memory* (the demonstration pool) while model parameters remain frozen, yielding no catastrophic forgetting but no generalisation beyond retrieval.

Limitations and generalizability. Queues with <50 demonstrations show degraded performance. Approximately 15% of errors involve external context inaccessible to text-based classification. The current mechanism captures only the first analyst action; secondary queue movements are not tracked. The approach requires queue-based routing, analyst review within the confirmation window, and visibility into corrections. Cross-organisational validation remains future work.

Deployment recommendations. Start in recommendation-only mode; identify high-confidence queues through production observation; expand automation gradually while monitoring override rates; maintain human override authority.

6 Conclusion

We presented an adaptive ticket routing system deployed at CERT-EU that frames routing as retrieval-augmented in-context learning with continual adaptation from analyst feedback. The system dynamically retrieves similar resolved tickets as demonstrations, enabling evidence-grounded classification from accumulated institutional expertise without requiring periodic retraining or explicit annotation.

Through 6 months of production deployment, the system achieves 86.3% weighted F1—outperforming supervised classification alternatives by +4.5pp and ungrounded LLM inference by +15pp. The substantial gap validates our architecture: dynamically retrieved demonstrations provide essential organisational context that general reasoning cannot replicate. A three-phase deployment strategy (bootstrapping → assisted routing → selective autonomy)

enabled progressive automation, achieving 5.5× overall speedup and 22× for selectively autonomous routing.

Key contributions: (1) temporal feedback learning that extracts learning signals from correction timing at zero incremental annotation cost—a novel approach that transforms routine workflow into continual model improvement without dedicated labelling effort; (2) retrieval-augmented in-context learning for classification with continuously evolving demonstrations and constrained outputs preventing invalid queue predictions; (3) phased deployment with empirical calibration for high-stakes environments; (4) production validation showing retrieval-augmented ICL outperforms supervised baselines while adapting continuously without retraining.

Unlike systems that ossify after deployment, our approach perpetually adapts while preserving institutional knowledge—enabling organisations to operationalise the tacit expertise of experienced analysts and maintain it across team transitions. While deployed in a cybersecurity context, the approach generalises to any domain with queue-based ticket routing under data-sovereignty constraints.

Future work. Several directions merit investigation: (1) extending temporal feedback to capture secondary queue movements for richer learning signals; (2) exploring whether domain-adapted embeddings could improve retrieval quality for our cybersecurity use case, given the specialised vocabulary of the field; (3) handling tickets that legitimately span multiple queues through multi-label prediction or suggested secondary assignments; (4) developing principled methods to dynamically adjust automation scope based on real-time performance metrics.

Ethical considerations

This work was conducted by the AI team at CERT-EU as part of operational improvements to cybersecurity services. The complete system runs entirely on-premise—no ticket data is transmitted to external services. Analysts retain override authority for all routing decisions, and autonomous routing is limited to empirically validated high-confidence queues. The views expressed are those of the authors and do not necessarily represent the official position of CERT-EU or any EU institution.

Acknowledgments

We thank the CERT-EU analyst team for their collaboration during system deployment and their feedback on the recommendation interface.

AI Assistance Disclosure. In accordance with KDD 2026 submission guidelines, we disclose that large language models were used to assist with editing and refining portions of this manuscript’s prose. The core technical contributions, system design, implementation, evaluation methodology, experimental results, and all scientific claims are entirely the work of the authors. The AI assistance was limited to improving clarity and readability of author-written content.

References

- [1] Saleema Amershi, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. 2019. Software Engineering for Machine Learning: A Case Study. In *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 291–300. doi:10.1109/ICSE-SEIP.2019.00042

- [2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. 1877–1901.
- [3] Huiyao Chen, Yu Zhao, Zulong Chen, Mengjia Wang, Liangyue Li, Meng Zhang, and Ming Zhang. 2024. Retrieval-style In-Context Learning for Few-shot Hierarchical Text Classification. *Transactions of the Association for Computational Linguistics* 12 (2024), 1–17.
- [4] Lixian Chen, Jie Wang, Huaibo Ma, and Wei Liu. 2012. Intelligent IT Support via Question-Answer Pair Recommendations. In *Proceedings of the 21st International Conference on World Wide Web (WWW)*. 597–598.
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. 4171–4186. doi:10.18653/v1/N19-1423
- [6] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Zhiyong Wu, Baobao Chang, Xu Sun, Jingjing Xu, Lei Li, and Zhifang Sui. 2023. A Survey on In-Context Learning. *arXiv preprint arXiv:2301.00234* (2023).
- [7] Aleksandra Edwards and Jose Camacho-Collados. 2024. Language Models for Text Classification: Is In-Context Learning Enough?. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING)*. 10058–10072.
- [8] Cristiano Mesquita Garcia, Ramon Simoes Abilio, Alessandro Lameiras Koerich, Alceu de Souza Britto Jr., and Jean Paul Barddal. 2024. Concept Drift Adaptation in Text Stream Mining Settings: A Systematic Review. *ACM Transactions on Intelligent Systems and Technology* (2024). doi:10.1145/3704922
- [9] Ian J. Goodfellow, Mehdi Mirza, Da Xiao, Aaron Courville, and Yoshua Bengio. 2013. An Empirical Investigation of Catastrophic Forgetting in Gradient-Based Neural Networks. *arXiv preprint arXiv:1312.6211* (2013).
- [10] Zeyu Han, Chao Gao, Jinyang Liu, Jeff Zhang, and Sai Qian Zhang. 2024. Parameter-Efficient Fine-Tuning for Large Models: A Comprehensive Survey. *arXiv preprint arXiv:2403.14608* (2024).
- [11] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *Proceedings of the International Conference on Learning Representations (ICLR)*. doi:10.48550/arXiv.2106.09685
- [12] Gautier Izacard and Edouard Grave. 2021. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*. 874–880. doi:10.18653/v1/2021.eacl-main.74
- [13] Leif Jonsson, Markus Borg, David Broman, Kristian Sandahl, Sigrid Eldh, and Per Runeson. 2016. Automated Bug Assignment: Ensemble-based Machine Learning in Large Scale Industrial Contexts. *Empirical Software Engineering* 21, 4 (2016), 1533–1578. doi:10.1007/s10664-015-9401-9
- [14] Vladimir Karpukhin, Barlas Ögüz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 6769–6781. doi:10.18653/v1/2020.emnlp-main.550
- [15] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 39–48. doi:10.1145/3397271.3401075
- [16] Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. 2022. A Continual Learning Survey: Defying Forgetting in Classification Tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 7 (2022), 3366–3385. doi:10.1109/TPAMI.2021.3057446
- [17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. 9459–9474.
- [18] Xiaonan Li, Kai Lv, Hang Yan, Tianyang Lin, Wei Zhu, Yuan Ni, Guotong Xie, Xiaoling Wang, and Xipeng Qiu. 2023. Unified Demonstration Retriever for In-Context Learning. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. 4644–4668. doi:10.18653/v1/2023.acl-long.256
- [19] Jiachang Liu, Dinghan Shen, Yizhe Zhang, Bill Dolan, Lawrence Carin, and Weizhu Chen. 2022. What Makes Good In-Context Examples for GPT-3?. In *Proceedings of Deep Learning Inside Out (DeeLIO): The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures, NAACL*. 100–114. doi:10.18653/v1/2022.deeLIO-1.10
- [20] Yao Lu, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp. 2022. Fantastically Ordered Prompts and Where to Find Them: Overcoming Few-Shot Prompt Order Sensitivity. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*. 8086–8098. doi:10.18653/v1/2022.acl-long.556
- [21] Eduardo Mosqueira-Rey, Elena Hernández-Pereira, David Alonso-Ríos, José Bobes-Bascarán, and Ángel Fernández-Leal. 2023. Human-in-the-loop Machine Learning: A State of the Art. *Artificial Intelligence Review* 56, 4 (2023), 3005–3054. doi:10.1007/s10462-022-10246-w
- [22] Elham Nouri and Christopher Gadowry. 2015. Real-Time Incident Ticket Routing Using Machine Learning. In *IEEE International Conference on Data Mining Workshop (ICDMW)*. 1053–1057.
- [23] Keqin Peng, Liang Ding, Yancheng Yuan, Xuebo Liu, Min Zhang, Yuanxin Ouyang, and Dacheng Tao. 2024. Revisiting Demonstration Selection Strategies in In-Context Learning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 9090–9101. doi:10.18653/v1/2024.acl-long.492
- [24] Arjun Rao, Hanieh Alipour, and Nick Pendar. 2025. Rethinking Hybrid Retrieval: When Small Embeddings and LLM Re-ranking Beat Bigger Models. *arXiv preprint arXiv:2506.00049* (2025).
- [25] Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. 2017. Snorkel: Rapid Training Data Creation with Weak Supervision. *Proceedings of the VLDB Endowment* 11, 3 (2017), 269–282. doi:10.14778/3157794.3157797
- [26] Alexander J. Ratner, Christopher M. De Sa, Sen Wu, Daniel Selsam, and Christopher Ré. 2016. Data Programming: Creating Large Training Sets, Quickly. In *Advances in Neural Information Processing Systems*, Vol. 29. 3567–3575.
- [27] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3, 4 (2009), 333–389. doi:10.1561/15000000019
- [28] Ohad Rubin, Jonathan Herzig, and Jonathan Berant. 2022. Learning to Retrieve Prompts for In-Context Learning. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL)*. 2655–2671. doi:10.18653/v1/2022.naacl-main.191
- [29] Pascal J. Sager, Ashwini Kamaraj, Benjamin F. Grewe, and Thilo Stadelmann. 2025. Deep Retrieval at CheckThat! 2025: Identifying Scientific Papers from Implicit Social Media Mentions via Hybrid Retrieval and Re-Ranking. *arXiv preprint arXiv:2505.23250* (2025).
- [30] Burr Settles. 2009. *Active Learning Literature Survey*. Technical Report 1648. University of Wisconsin-Madison, Department of Computer Sciences.
- [31] Haizhou Shi, Zihao Xu, Hengyi Wang, Weiyei Qin, Wenyuan Wang, Yibin Wang, Zifeng Wang, Sayna Ebrahimi, and Hao Wang. 2024. Continual Learning of Large Language Models: A Comprehensive Survey. *arXiv preprint arXiv:2404.16789* (2024).
- [32] Pannaga Shivaswamy and Thorsten Joachims. 2015. Coactive Learning. *Journal of Artificial Intelligence Research* 53 (2015), 1–40. doi:10.1613/jair.4539
- [33] Mohammed Latif Siddiq and Joanna C. S. Santos. 2022. BERT-Based GitHub Issue Report Classification. In *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering (NLBSE), co-located with ICSE*. 33–36. doi:10.1145/3528588.3528660
- [34] Aaron D. Tucker, Kianté Brantley, Adam Cahall, and Thorsten Joachims. 2024. Coactive Learning for Large Language Models using Implicit User Feedback. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. 48809–48822.
- [35] Liyuan Wang, Xingxing Zhang, Hang Su, and Jun Zhu. 2024. A Comprehensive Survey of Continual Learning: Theory, Method and Application. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46, 8 (2024), 5362–5383. doi:10.1109/TPAMI.2024.3367329
- [36] Xingjiao Wu, Luwei Xiao, Yixuan Sun, Junhang Zhang, Tianlong Ma, and Liang He. 2022. A Survey of Human-in-the-Loop for Machine Learning. *Future Generation Computer Systems* 135 (2022), 364–381. doi:10.1016/j.future.2022.05.014
- [37] Han Yuan, Liang Kang, Yiming Li, and Zhuo Fan. 2024. Human-in-the-loop Machine Learning for Healthcare: Current Progress and Future Opportunities in Electronic Health Records. *Medicine Advances* 2, 3 (2024), 318–322. doi:10.1002/med4.70
- [38] Alessandro Zangari, Matteo Marcuzzo, Michele Schiavinato, Andrea Gasparetto, and Andrea Albarelli. 2023. Ticket Automation: An Insight into Current Research with Applications to Multi-level Classification Scenarios. *Expert Systems with Applications* 225 (2023), 119984. doi:10.1016/j.eswa.2023.119984

A System Configuration Details

A.1 Hyperparameter Configuration

Table 6 lists the key hyperparameters used in the deployed system.

Table 6: System hyperparameters and rationale.

Parameter	Value	Rationale
K (demonstrations)	5	Context
Prefetch candidates	20	Pool for reranking
Dense embedding dim	768	nomic-embed-text default
Prefetch weights (α, β)	0.5, 0.5	Equal dense/sparse
Reranking	ColBERT v2	Late interaction
Correction window	7 days	95% corrections captured
Confirmation threshold	8 days	Buffer after corrections
Redis TTL	10 days	Late confirmation buffer
Autonomy F1 threshold	0.86	Empirical cutoff

A.2 Inference Latency

Table 7 shows end-to-end inference latency by pipeline path. The consensus path applies when all retrieved demonstrations agree, bypassing LLM inference entirely and achieving sub-second latency. The remaining require LLM conflict resolution.

Table 7: Inference latency by pipeline path. The consensus path applies when all K demonstrations agree (no LLM needed); the LLM path applies when demonstrations disagree.

Operation	Median	P95
Embedding generation	45ms	78ms
Vector retrieval ($K = 5$)	15ms	32ms
ColBERT reranking	85ms	142ms
LLM conflict resolution	15.2s	22.4s
Total (consensus path)	~150ms	~250ms
Total (LLM path)	~15.5s	~23.0s

A.3 Infrastructure

Table 8 summarises the complete system stack for reproducibility.

Table 8: System stack. All inference runs on-premise.

Component	Technology	Details
Hardware	Mac Studio M3 Ultra	512GB unified memory
LLM	Qwen3:32B	Full precision, llama.cpp
Dense Emb.	nomic-embed-text	768d, llama.cpp
Sparse Emb.	BM25	Lexical ranking
Reranking	ColBERT v2	Late interaction
Vector DB	Qdrant	Single node
Monitoring	Redis	10-day TTL
Queue	Apache Kafka	Ingestion pipeline

B Threshold Sensitivity Analysis

The 7-day correction window and 8-day confirmation threshold are central parameters of the temporal feedback mechanism. We analysed sensitivity to these thresholds using production data from the full 6-month deployment.

Correction window analysis. Table 9 shows the cumulative distribution of analyst correction times.

The 7-day threshold captures 95% of corrections while keeping the observation window practical. Shorter windows (3–5 days) miss

Table 9: Cumulative analyst corrections captured by threshold.

Days after prediction	Cumulative corrections captured
3	78%
5	89%
7 (chosen)	95%
10	98%
14	99%

11–22% of corrections, potentially indexing incorrect predictions as confirmations. Longer windows (10–14 days) capture marginally more corrections but delay the confirmation signal, reducing the system’s adaptation speed.

Impact of threshold on system quality. Based on internal analysis, the majority of tickets are triaged within the first few days of arrival—approximately 80% of all routing decisions (corrections or confirmations) occur within 5 business days. The 7-day window thus covers most operational activity including weekends and holidays.

Confirmation threshold. The 8-day confirmation threshold (one day after the correction window) provides a buffer that accounts for edge cases such as analysts being out of office. In practice, we observed that extending the confirmation threshold to 10 or 14 days changed fewer than 2% of labels, indicating the 8-day threshold is robust.

C Statistical Significance Testing

To confirm that the performance differences reported in Table 1 are not due to chance, we performed paired McNemar’s χ^2 tests (continuity-corrected) for each baseline against our system on the same 1900-ticket evaluation set. Our system achieves 86.1% accuracy (1636/1900 correct). Table 10 reports the results.

Table 10: McNemar’s χ^2 test (continuity-corrected) comparing each baseline against our system on 1900 evaluation tickets. $b+c$ denotes the number of discordant pairs.

System	Acc	$b+c$	χ^2	p
Ours (reference)	86.1	—	—	—
Zero-shot LLM	71.1	661	122.02	$< 10^{-15}$
KNN + sem.	74.8	609	75.20	$< 10^{-15}$
Few-shot LLM	75.4	601	67.89	2.2×10^{-16}
XGBoost + TF-IDF	77.8	568	43.40	4.5×10^{-11}
XGBoost + sem.	82.0	510	11.63	6.5×10^{-4}
LoRA fine-tuned	83.2	187	15.59	7.9×10^{-5}

All six differences are statistically significant at $p < 0.001$, confirming that the improvements reported in Table 1 are not due to chance.

D LLM Prompt Template

The following is the production prompt template used for queue prediction when retrieved demonstrations have conflicting queue assignments. This template is provided verbatim for reproducibility:

You are an expert ticket classification system. Your role is to accurately determine the most appropriate queue assignment for incoming tickets based on ticket content analysis and historical patterns.

CLASSIFICATION PROCESS:

1. Content Analysis: Thoroughly examine the new ticket's subject, description, and details
2. Historical Pattern Recognition: Analyse similar historical tickets, their queue assignments, and similarity scores
3. Similarity Weighting: Prioritise tickets with higher similarity scores
4. Queue Selection: Choose the most appropriate queue from the available options based on content match and historical precedent

DECISION CRITERIA:

- Ticket type and queue (security tickets, administrative procedures, HR matters, general enquiries, etc.)
- Subject matter and content themes
- Required expertise and handling procedures
- Historical queue assignment patterns for similar tickets
- Organisational workflow and departmental responsibilities

IMPORTANT CONSTRAINTS:

- You MUST select a queue that appears in the similar tickets provided below
- Do NOT create or suggest new queue names
- Base your decision on the available queue options from the historical data

SIMILAR TICKETS FOR REFERENCE:

{similar_tickets}

TICKET TO CLASSIFY:

{ticket_to_classify}

OUTPUT REQUIREMENTS:

Respond with ONLY the queue name that exists in the similar tickets above. No explanations, justifications, or additional text.

CRITICAL: Your answer must be one of the queue names from the similar tickets provided.

The {similar_tickets} placeholder is populated with the retrieved demonstrations, each formatted with queue assignment, similarity score, and ticket subject. The {ticket_to_classify} placeholder contains the new ticket's subject and body text. The constraint to select only from queues present in the demonstrations is enforced both in the prompt and through post-processing validation.